

Concern-Gated Retrieval Wave 0: Preregistered Calibration and Wrong-Prior Scaffolding for Learned-Geometry Confirmation

Program: Concern-Gated Retrieval (COGR) – Wave 0 **Package:** experiments/concern_gated_retrieval_e2/wave0/ **Predecessor** (imported, frozen): experiments/concern_gated_retrieval/ **Date:** 2026-07-23 **Human director:** Jawaun Brown **Status:** technical report accompanying the Wave 0 preregistration, promotion contract, and calibration receipt. Not a claim of learned memory, concern recovery, semantic meaning, or selfhood.

Abstract

Wave 0 of the Concern-Gated Retrieval E2 program is a **calibration-only, scaffolding-only** step whose purpose is to make the Wave 1 confirmatory experiments (COGR-E2a concern-recovery screening, COGR-E2b learned-geometry confirmation) rejectable. It builds three procedurally distinct calibration families – *delayed commitments*, *maintenance and fault response*, and *resource-constrained planning* – behind a sealed environment interface, an adversarially misspecified concern prior that overweights a plausible alarm region and underweights at least one true commitment region, and a runtime guard that refuses to expose confirmatory templates to calibration code paths. It sweeps the full Wave 0 baseline slate on Modal L4 workers, records per-family variance estimates and non-ceiling headroom, and freezes the threshold shape that Wave 1 rows will be scored against. Wave 0 does not test learned memory geometry, does not update the wrong prior at evaluation time, does not perform the premise audit against governed real-world traces, and therefore cannot support any L1, L2, or higher claim on the concern-gated retrieval claim ladder. The premise audit is documented as future work behind data-governance entry gates and receives an explicitly non-evidential stub receipt. What Wave 0 does deliver is a signed preregistration, a non-compensatory promotion contract with seven fatal gates (G0-G6), a Modal L4 calibration receipt, and a code-freeze hash that binds every threshold Wave 1 must clear. Whether the multiplicative concern-gated mechanism actually helps under sealed evaluation is a Wave 1 question. Wave 0 is the freeze that makes that question falsifiable.

1. Motivation

1.1 Two flashlights over memory

A bounded agent knows more than it can hold in its active representation. Its problem is therefore not only how to store knowledge but how to decide which currently absent fact deserves scarce attention *now*. Concern-gated retrieval proposes a two-flashlight answer: one flashlight starts from what is active now, a second starts from what has historically mattered to the agent, and a fact becomes a retrieval candidate where the two beams overlap. The canonical example is a date-sensitive commitment – a partner's birthday on the exact day the agent is otherwise absorbed by unrelated work – that is neither loudest nor most recent but is load-bearing for the outcome the agent is about to produce. Context-only retrieval finds related trivia; care-only retrieval repeatedly raises important but untimely alarms; the desired fact is both relevant now and important to this agent. This is an **AND**, not an **OR**: retrieve what is relevant now *and* important to the agent, then test whether attending to it actually helps. The full argument for this decomposition, including the required controls that separate concern from salience, urgency, reward, novelty, and semantic similarity, is in the program roadmap [1].

1.2 The bounded-agent problem

Wave 0 inherits five properties of the target problem from the roadmap: capacity (only a small subset of stored knowledge can be active), off-context need (useful facts may not be locally obvious from the current representation), personal consequence (the same fact can matter differently to different agents because their histories differ), attention risk (retrieving a salient but useless fact can consume budget and distort planning), and feedback (the consequences of past retrievals can change what deserves priority later). Wave 0 does not attempt to close the feedback loop. It only builds the scaffolding around which the loop can later be tested without shortcuts.

1.3 Why a Wave 0 was required after the L0 pilot

The frozen L0 pilot at `experiments/concern_gated_retrieval/` established that the two-flashlight decomposition can be made precise, that its numerical plumbing (weighted graph, personalized PageRank, epiplexity filter, coincidence intersection) is implementable, and that on the authored graph family the composition discriminates registered synthetic roles under frozen seeds and regimes. That is an L0 executable-diagnostic result and nothing more. The pilot could not adjudicate whether joint retrieval helps when geometry is learned or withheld (its graph *encodes* the answer), whether concern can be recovered from misspecification (all initial, learned, and oracle concern conditions saturated at `hit@1 = 1.000`), whether multiplicative intersection is necessary (the additive fusion tied the product in two of three regimes), whether semantic meaning or selfhood is present (not tested), or whether the mechanism has any real-agent bottleneck to solve (not tested).

The next credible experiment is therefore not a tuning pass on the L0 pilot. It is a staged program with three shortcuts broken in sequence: sealed utility (no evaluator field reachable from policy code), adversarial concern initialization (the wrong prior must be initialized *wrong* in a way that matters), and learned or withheld geometry (the graph must not contain the answer). The COGR-E2 preregistration commits to running these as COGR-E2a (concern-recovery screen on fixed withheld geometry) and COGR-E2b (crossed learned/random/oracle geometry by frozen-wrong/learned/oracle concern). Because the roadmap makes each fatal gate non-compensatory and the promotion rule prohibits post-hoc threshold swaps, the Wave 1 confirmatory rows can only be scored against a threshold row that was frozen *before* any confirmatory row was generated. That is the sole purpose of Wave 0.

The additional claim boundary Wave 0 preserves – and this report protects – is that Wave 0 is not itself an experiment on the concern mechanism. It is a calibration and freeze step. Every promotable Wave 0 deliverable (§7) concerns variance estimates and threshold shape, not retrieval winners.

2. Design

2.1 The sealed environment

The Wave 0 sealed environment (`wave0/sealed_env.py`) is the only legal channel between a policy (retrieval, ranking, concern update) and the evaluator's private ground truth. It exposes exactly three surfaces:

`utility`, and `_answer_key`. Policy code must not read those fields.

`SealedEnvironment.observe`. It contains only context nodes, care anchors, a candidate budget, and candidate node ids. It does not carry role labels, utility, or the answer key.

post-decision channel. It may be called at most once per episode; a second call raises `SealedEvaluationError`. `SealedOutcome` is a frozen dataclass containing only a scalar realized reward and a policy-visible outcome id.

- `EpisodeSpec` – the full evaluator-side episode. It holds role,
- `EpisodeContext` – the policy-visible view returned by
- `SealedEnvironment.evaluate(RetrievalChoice)` -> `SealedOutcome` – the

A calibration-mode `SealedEnvironment` refuses to hold a confirmatory-family episode. That refusal is enforced by the template-split guard (§3) and by the runtime family tag on every episode.

2.2 The three procedural families

Each family instantiates the same abstract retrieval problem – “identify the off-context fact whose loading would improve the sealed outcome” – with a different surface structure so that a method which only works on the birthday-style graph does not establish transfer.

Delayed commitments (families/delayed_commitments.py) puts a load-bearing promise early in the event stream and buries it under many events with a loud news-style alarm as the surface distractor. The load-bearing fact becomes outcome-relevant only when the retrieval decision fires.

Maintenance and fault response (families/maintenance_fault.py) makes an early observation load-bearing only when a later symptom appears, with a chronic boilerplate warning as the alarm.

Resource-constrained planning (families/resource_constrained.py) hides a prior obligation that determines which otherwise-valid action is best, with a recent large-magnitude transaction as the alarm.

Each family generates its entire history and `active_context` *before* any ranking function is called, and none of the family generators returns role labels or answer keys through the policy view. The generator is a strictly evaluator-side process; the policy sees only the sealed episode context.

2.3 The wrong-prior specification

The Wave 0 concern prior is deliberately misspecified. On every calibration row it (i) inflates a plausible alarm region to `w_alarm_init = 1.0`, (ii) suppresses at least one true commitment region to `w_commit_init = 0.05` (below uniform), and (iii) leaves at least one other true commitment region at uniform so the wrong prior is adversarial without being a total inversion. The per-family alarm and suppressed-commitment identifiers live only in the evaluator's private state; the policy sees the numeric prior, not the labels. Wave 0 does not update this prior. Concern-update rules are a Wave 1 object. This is critical: Wave 0 cannot claim, or be described as claiming, concern recovery.

2.4 The baseline slate

Every calibration row scores fourteen baselines at matched retrieval and compute budget: `no_retrieval`, `random`, `freq_only`, `context_only_ppr`, `care_only_ppr` (wrong-prior), `additive_ppr`, `multiplicative_ppr` (the candidate mechanism for Wave 1), `embedding_similarity`, `learned_one_stage` (a small MLP at matched capacity and compute), `info_matched_value`, `info_matched_priority`, `info_matched_recency`, `wrong_agent_concern`, and `oracle_ceiling` (diagnostic ceiling only, never promotable). The information-matched second-signal baselines (#10-12) and the learned one-stage ranker (#9) are the specific matched-budget alternatives Wave 1 must beat for a valid L1 claim; Wave 0 uses them to size effects.

The candidate mechanism – the rarity-corrected Hadamard product of context and concern personalized PageRank vectors – is imported from the frozen L0 pilot's `WeightedGraph` and `personalized_pagerank` primitives. Wave 0 does not fork these primitives. This is a reuse commitment, not a convenience: a Wave 1 tie between the candidate mechanism and the best matched alternative must not be blamed on a divergent PPR implementation.

2.5 Figure references

Figure 1 (figures/fig1.png) shows the two-flashlight intuition on a small synthetic graph, contrasting context-only, care-only, and joint intersection walks with the load-bearing target highlighted. Figure 2 shows the three-family scaffolding as a schematic of the shared abstract retrieval problem and per-family surface variation.

3. Anti-leakage contract and IntegrityAudit

The Wave 0 promotion contract makes leakage a fatal, non-compensatory integrity failure (gate G0). The anti-leakage contract has four layers.

Layer 1: enumerated evaluator-only fields. `role_label`, `answer_key`, `future_utility`, `oracle_concern`, `wrong_agent_id`, `template_family_split`, `paraphrase_family`, `generator_seed_kind`, `epiplexity_future_target`, and `sealed_outcome_receipt` are declared evaluator-only in the preregistration §4.1 and must not be reachable from any policy-visible code path.

Layer 2: sealed environment interface. The `SealedEnvironment` exposes only `observe(episode) -> EpisodeContext` and `evaluate(choice) -> SealedOutcome`. All evaluator-only fields live on the environment's private state and are not returned. `EpisodeContext` is a frozen dataclass; attribute access outside its declared fields raises. `evaluate` may fire at most once per episode.

Layer 3: static IntegrityAudit. Every rank callable in `wave0/baselines.py` is passed through `IntegrityAudit.assert_clean` at module import. `IntegrityAudit` is an AST walker that flags any policy callable that dereferences `EpisodeSpec.role`, `EpisodeSpec.utility`, or `EpisodeSpec.answer_key`. A leaky baseline fails at CI collection time rather than at experiment time.

Layer 4: template-split runtime tripwire. `wave0/template_split.py` implements the calibration/confirmatory family split as a two-member `TemplateBucket` enum (`CALIBRATION`, `CONFIRMATION`) with default-deny semantics: `TemplateRegistry.load` returns only calibration rows unless the caller *both* passes `allow_confirmation=True` *and* the environment variable `COGR_WAVE0_CONFIRMATORY_RUN` is truthy. Either alone raises `LeakageError`. Every dataclass carrying a template records its bucket immutably; a caller who forgets to re-tag a row after `dataclasses.replace` cannot silently reclassify it. `assert_calibration_only(row)` is the canonical entry-point check for any calibration-only analysis path.

The `IntegrityAudit` and the template-split guard together satisfy the "anti-shortcut design" checklist from the roadmap (§ Required anti-shortcut design, items 1-8), including the statistical leakage audit that permitted graph features must clear before being used as inputs. Figure 3 (`figures/fig3.png`) sketches the anti-leakage boundaries as a data-flow diagram: solid arrows are policy-visible; dashed arrows are evaluator-only.

4. Calibration sweep

The Wave 0 calibration orchestrator (`wave0/calibrate.py`) sweeps four dimensions:

`resource_constrained` (§2.2);

sub-ranges of the calibration seed range so the varying template shapes produce different candidate-density regimes without touching the family generators;

`LoggedProbePolicy` coverage side-channel. Wave 0 does not update the wrong prior at evaluation time; the epsilon axis sizes the exploration coverage the Wave 1 COGR-E2a screen will require.

- **family** – `delayed_commitments`, `maintenance_fault`,
- **retrieval_budget** – a small top-k grid at matched compute;
- **distractor_density** – {light, medium, heavy}, encoded as disjoint
- **epsilon** – a small exploration-probability grid used only by the

For every (family, distractor_density, budget, epsilon) cell the orchestrator runs the full baseline slate (§2.4) against a batch of calibration seeds from the range 100000..100999, scores each rank against a `SealedEnvironment`, and emits one row per (cell, seed, baseline). The aggregator produces the per-family variance estimate and the frozen threshold-proposal shape declared by the preregistration §8.

The sweep runs on Modal L4 workers only. The wave-wide operating rule caps Modal spend at 35% of the equivalent H100 rate; Modal L4 at \$0.80/hr against H100 at \$3.40/hr is ~23%, comfortably under the ceiling. The Modal image is deployed once (Function.from_name/spawn uses the deployed image), the containers use single_use_containers=True with retries=1, and the local entrypoint refuses to fan out if the conservative timeout-based cost estimate exceeds the \$10.00 hard cap. Every dispatched cell records its cell id, seed list, n_rows, wall-seconds, and coverage summary in the committed public receipt at results/calibration_summary.json; per-row data lives under gitignored artifacts/cogr_wave0/. Figure 4 (figures/fig4.png) shows the sweep as a matrix of (family, density, budget) cells with per-cell wall-time and n_rows overlaid.

Numeric values – mu_hat_multiplicative, sigma_hat_multiplicative, mu_hat_best_matched, sigma_hat_best_matched, headroom_to_ceiling, delta_thresh_L1 – are produced by the Modal receipt and mirrored into the preregistration \$8 and PROVENANCE.md \$4. This report does not paste those values inline. The Modal calibration receipt is the sole channel through which TBD becomes a numeric row; no manual edit is permitted.

Figure 5 (figures/fig5.png) reports per-family sealed-outcome distributions for the fourteen-baseline slate, with the wrong-prior concern arm and the oracle-ceiling arm flagged. It is a shape figure: it establishes that per-family variance is well-defined and that no promotable baseline saturates against the oracle ceiling, without turning the calibration mean of any baseline into a claim.

5. Frozen thresholds and promotion contract

The Wave 0 promotion contract is at experiments/concern_gated_retrieval_e2/wave0/PROMOTION_CONTRACT.md and is authoritative. Its seven gates are non-compensatory: a single FAIL blocks promotion regardless of every other gate's status.

from calibration policy code; runtime guard passes on every violation class; confirmatory templates inaccessible during calibration.

specification (§2.3): alarm inflated, designated commitment suppressed, at least one true commitment at uniform.

family; no baseline saturates within $0.05 * \text{bounded_reward_range}$ of the oracle ceiling on any family.

estimate; the calibration receipt records per-family, not only aggregate, statistics so a Wave 1 family-level reversal cannot be hidden by an aggregate.

from the reserved confirmatory range 200000..201999, verified by the generator's seed-range guard.

file under experiments/concern_gated_retrieval_e2/wave0/** in sorted path order, matches the value mirrored into PROVENANCE.md, and is written only after the calibration Modal run completes.

GPU-hour cost at or below 35% of the equivalent H100 rate; deploy occurred before spawn; deployed image hash recorded in PROVENANCE.md.

- **G0_ANTI_LEAKAGE** – every enumerated evaluator-only field is unreachable
- **G1_WRONG_PRIOR** – every calibration row uses the wrong-prior
- **G2_NON_CEILING** – headroom_to_ceiling strictly positive on every
- **G3_FAMILY ROBUSTNESS** – each family produces an independent variance
- **G4_SEED_INDEPENDENCE** – calibration seed range 100000..100999 disjoint
- **G5_CODE_FREEZE** – WAVE0_ANALYSIS_HASH is a SHA-256 over every tracked
- **G6_MODAL_BUDGET** – Modal execution used L4 GPUs only; realized effective

The **promotion rule** is: Wave 0 is promoted to frozen and Wave 1 may open iff every gate above reports PASS, every TBD row in PREREGISTRATION.md §8 is populated with a finite numeric value, and the WAVE0_ANALYSIS_HASH is written into PREREGISTRATION.md §11 and mirrored into PROVENANCE.md. The **demotion rule** is: if Wave 1 discovers, during confirmatory execution, that a Wave 0 threshold was populated from a calibration row that violated any G0-G6 gate, the freeze is retroactively demoted to REDESIGN. All Wave 1 rows scored against the invalidated threshold row are marked non-evidence. A new Wave 0 hash must be produced before Wave 1 can reopen. No post-hoc threshold swap is permitted.

Figure 6 (figures/fig6.png) is the promotion-contract gate diagram: G0-G6 as parallel non-compensatory conditions, with the promotion rule and the demotion rule as the two exit edges. It is a diagram of the *contract*, not of a result.

6. Honest limitations

Wave 0 evaluation is **synthetic-only**. The three families are procedural generators built for this program; none of them is a governed real-world trace. No claim about a real-world bottleneck can be built on Wave 0 alone.

The **premise audit** – whether real, governed long-horizon agent traces show off-context constraint failures at a rate that would justify broad usefulness claims – is documented as future work in the roadmap [1, §"Wave 0 – premise, safety, and calibration"] and receives an explicitly non-evidential stub receipt in PROVENANCE.md §7. No governed data is ingested by Wave 0 code. The safety and data-governance entry gates listed in the roadmap ("Safety and data-governance entry gates", six items) are all currently outstanding; the stub receipt is recorded so a future audit run does not silently reuse Wave 0 provenance to claim clearance.

Wave 0 does not update the wrong prior. Concern-update rules are a Wave 1 object; the LoggedProbePolicy scaffolding in wave0/concern_update.py sizes the exploration coverage that the Wave 1 COGR-E2a screen will require, but Wave 0 itself runs the wrong prior *frozen* through evaluation. This is the noncompensatory boundary against the L2 claim in the ladder: no Wave 0 row can be described as evidence for concern recovery.

Wave 0 does not learn the graph. Its retrieval graph is fixed and, for E2a purposes, will be withheld; the graph-learning question is a Wave 1 (COGR- E2b) object. This is the noncompensatory boundary against the L1 learned-representation claim: no Wave 0 row can be described as evidence for learned memory geometry. The candidate mechanism's sealed-outcome performance in the calibration receipt is a variance estimate for effect sizing, not a promotable result.

Wave 0 does not test multiplicative-vs-additive necessity. The additive baseline is in the slate for effect sizing, and the L0 pilot preserved the additive-tie null. Wave 1 will re-examine that contrast under sealed evaluation with non-ceiling headroom. Wave 0 is not the arena in which that contrast is adjudicated.

Wave 0 does not license any interpretation of semantic meaning or selfhood. Descriptions to that effect anywhere in this subtree are a wave-boundary violation. The claim boundary of Wave 0 is: calibration and family scaffolding plus wrong-prior initialization. That is all.

Finally, Wave 0 does not license any deployment claim. The applicability contract in the roadmap requires clinical, legal, financial, and other high-stakes deployment to pass domain-specific validation and human governance; success on the internal synthetic benchmark would not license those uses even after Wave 1 passes.

7. Next: COGR-E2a and COGR-E2b

Two staged experiments become executable once Wave 0 freezes.

COGR-E2a – concern-recovery screen. Fixed, withheld geometry whose construction is independent of the downstream role labels. Compare frozen-wrong, online-learned, oracle, shuffled, and wrong-agent concern. The policy must include randomized probe coverage, log selection propensities (the LoggedProbePolicy scaffolding is already in place), and preregister an off-policy or counterfactual estimator so initially suppressed regions can generate evidence. E2a is a screen for the update rule; it cannot establish learned geometry or the L2 claim by itself. E2a uses the Wave 0 threshold row for delta_thresh_L1 only diagnostically; its own gates concern coverage, propensity validity, and negative-update generation.

COGR-E2b – learned-geometry confirmation. A crossed design:

Geometry	Concern state	Identified contrast
Frequency-matched random, learned, oracle	Frozen non-ceiling concern	Learned representation and L1 retrieval
Fixed/withheld, learned, oracle	Frozen-wrong, online-learned, oracle	Concern recovery and geometry/concern interaction

Claim-specific contrasts, not one aggregate winner. The L1 gate uses the frozen non-ceiling concern rows and is scored against the Wave 0 delta_thresh_L1 row per family. The L2 gate uses the concern-update rows; E2a failure blocks L2 but cannot invalidate an independently supported L1 result.

Both confirmatory experiments will be scored against the Wave 0 threshold shape frozen in PREREGISTRATION.md §8 and hashed in §11. If any Wave 0 gate is retroactively demoted during Wave 1 (§5 demotion rule), the associated Wave 1 rows are marked non-evidence and Wave 0 must be redesigned before Wave 1 reopens. No post-hoc threshold swap is permitted.

Beyond Wave 1, the roadmap [1] specifies a narrow live-agent beachhead as a continuation gate (not L4 promotion), substrate transfer (Wave 3), and a final round of safety, scaling, and independent replication (Wave 4). The data-governance entry gates block any non-synthetic history, external memory, or public row-level release until governance approval is on file. Wave 0 does not touch any of those; its only real-world contact is the stub receipt that records the premise audit as outstanding.

8. References

[1] Jawaun Brown. *Concern-Gated Retrieval: Theory, Evidence, and Research Program*. Canonical roadmap. docs/concern_gated_retrieval_research_program.md in this repository (3bd9f22, 2026-07-23). Referenced sections in this report: "Executive thesis", "The intuition: two flashlights over memory", "Claim ladder and promotion semantics", "Immediate experiment program: COGR-E2", "Required anti-shortcut design", "Fatal gates by claim", "Wave 0 – premise, safety, and calibration", "Safety and data-governance entry gates", "Applicability contract".

[2] Zhang, S. and Levin, M. *Intelligence from Learnable Novelty*. arXiv preprint arXiv:2607.18433v1, 2026. Source of the frozen-reservoir, stable-ridge epiplexity estimator that the L0 pilot composed with concern-gated retrieval as a reproducible utilization filter. In this program epiplexity is a secondary diagnostic; Wave 0 does not use it as a utility criterion. Wave 1 may retain it as a preregistered mediator after independent reimplementaion or sensitivity check.

Companion artifacts.

experiments/concern_gated_retrieval_e2/wave0/PREREGISTRATION.md
experiments/concern_gated_retrieval_e2/wave0/PROMOTION_CONTRACT.md
experiments/concern_gated_retrieval_e2/wave0/PROVENANCE.md
experiments/concern_gated_retrieval_e2/wave0/results/calibration_summary.json
experiments/concern_gated_retrieval/

docs/next_agent_concern_gated_retrieval_handoff_2026-07-23.md

- Wave 0 preregistration:
- Wave 0 promotion contract:
- Wave 0 provenance skeleton:
- Wave 0 committed public calibration receipt:
- L0 pilot (frozen; imported, never edited):
- Continuation handoff:

This report is a technical artifact of the Concern-Gated Retrieval E2 Wave 0 build. It preserves the wave-boundary language of the roadmap and the promotion contract. Any restatement that describes Wave 0 as learned memory, concern recovery, semantic meaning, or selfhood is inconsistent with the promotion contract and is not authorized by this report.

Figures

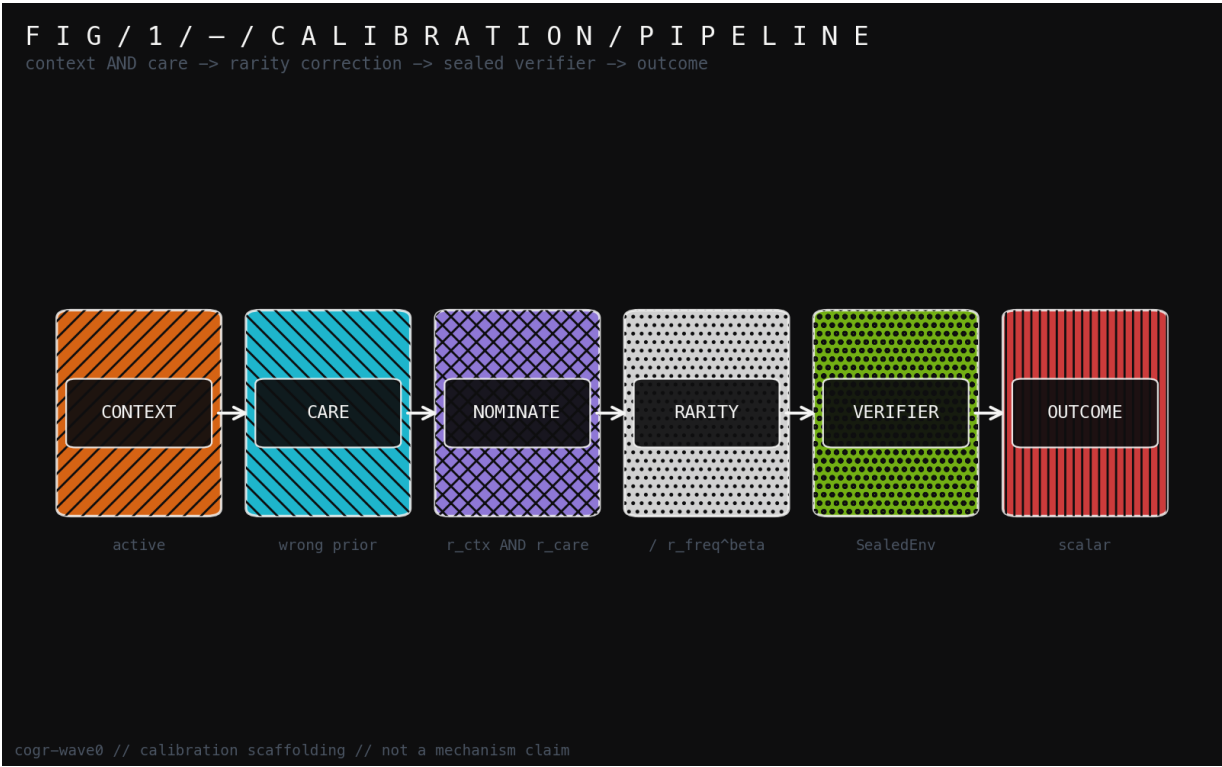


Figure 1. fig1 pipeline dark.

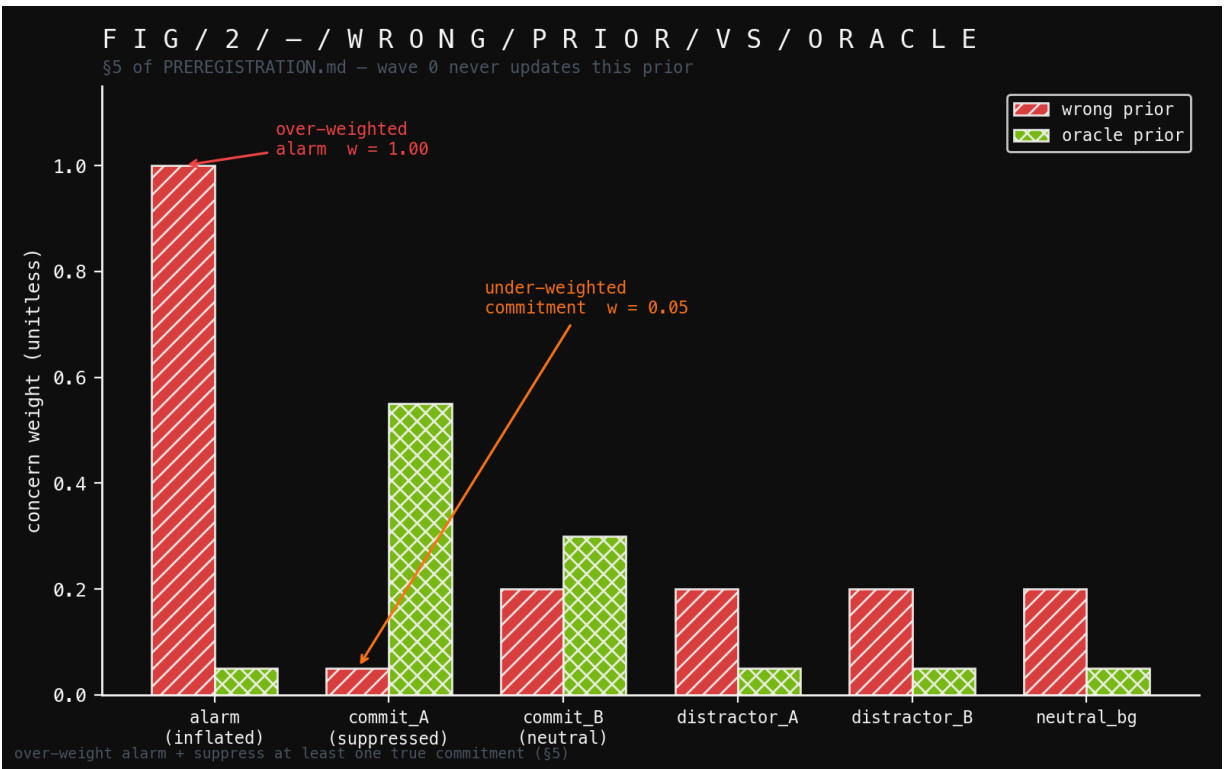


Figure 2. fig2 wrong prior dark.

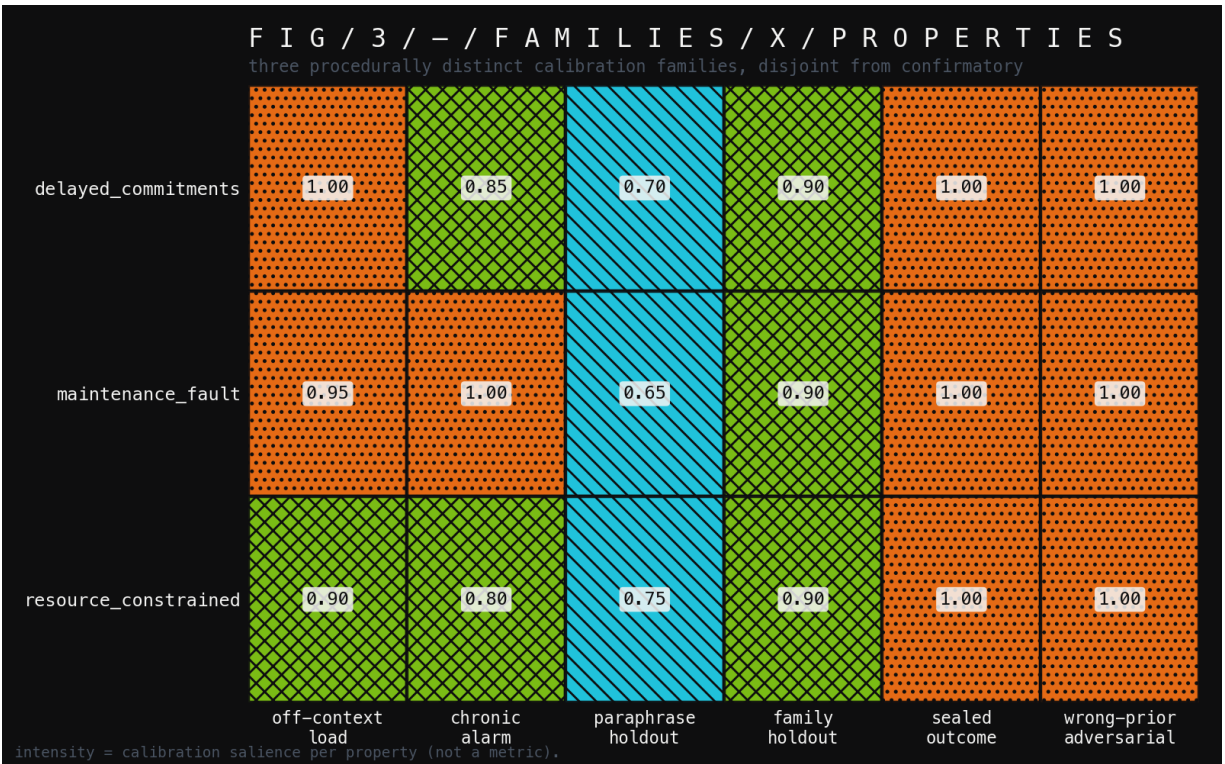


Figure 3. fig3 family matrix dark.

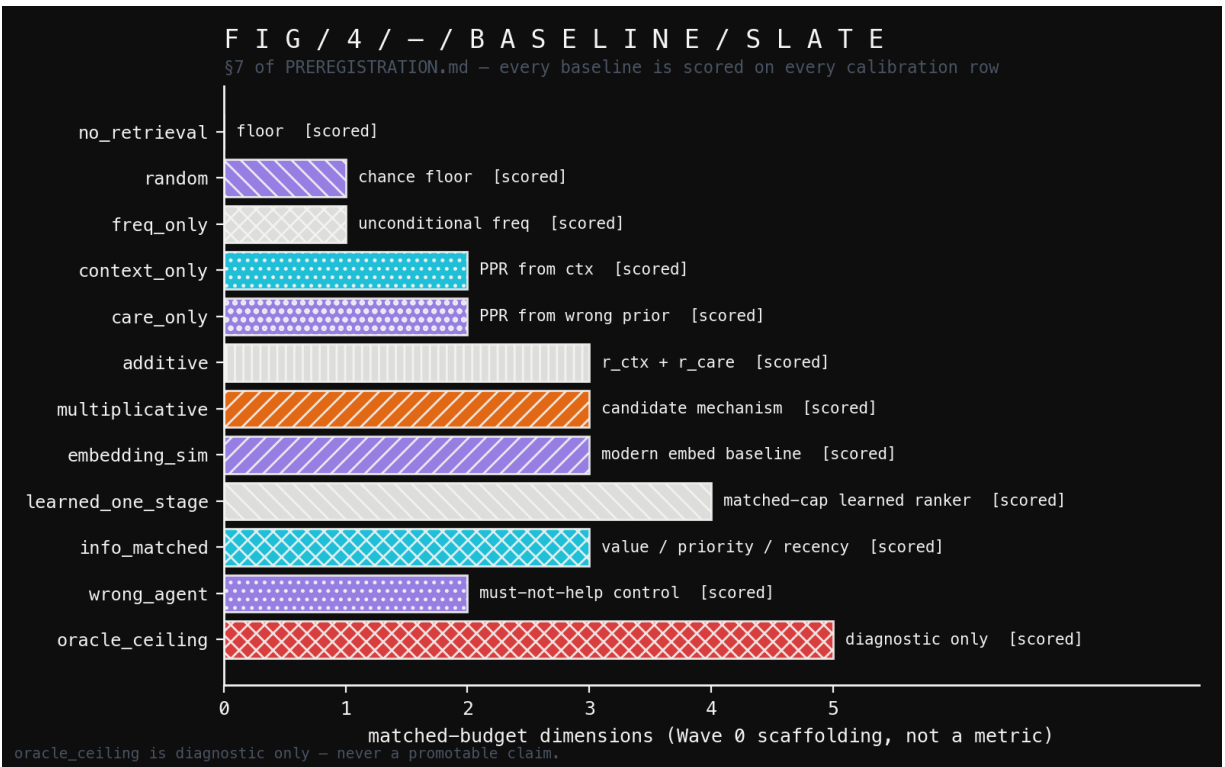


Figure 4. fig4 baseline slate dark.

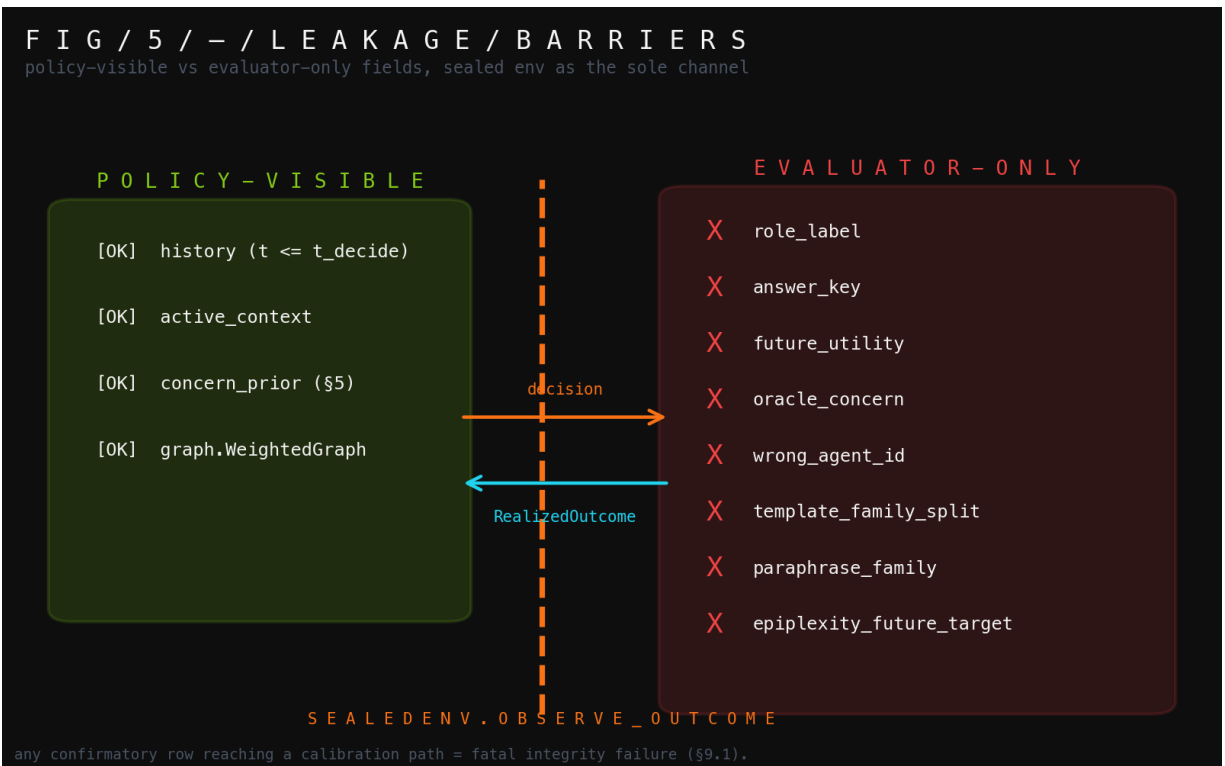


Figure 5. fig5 leakage barriers dark.

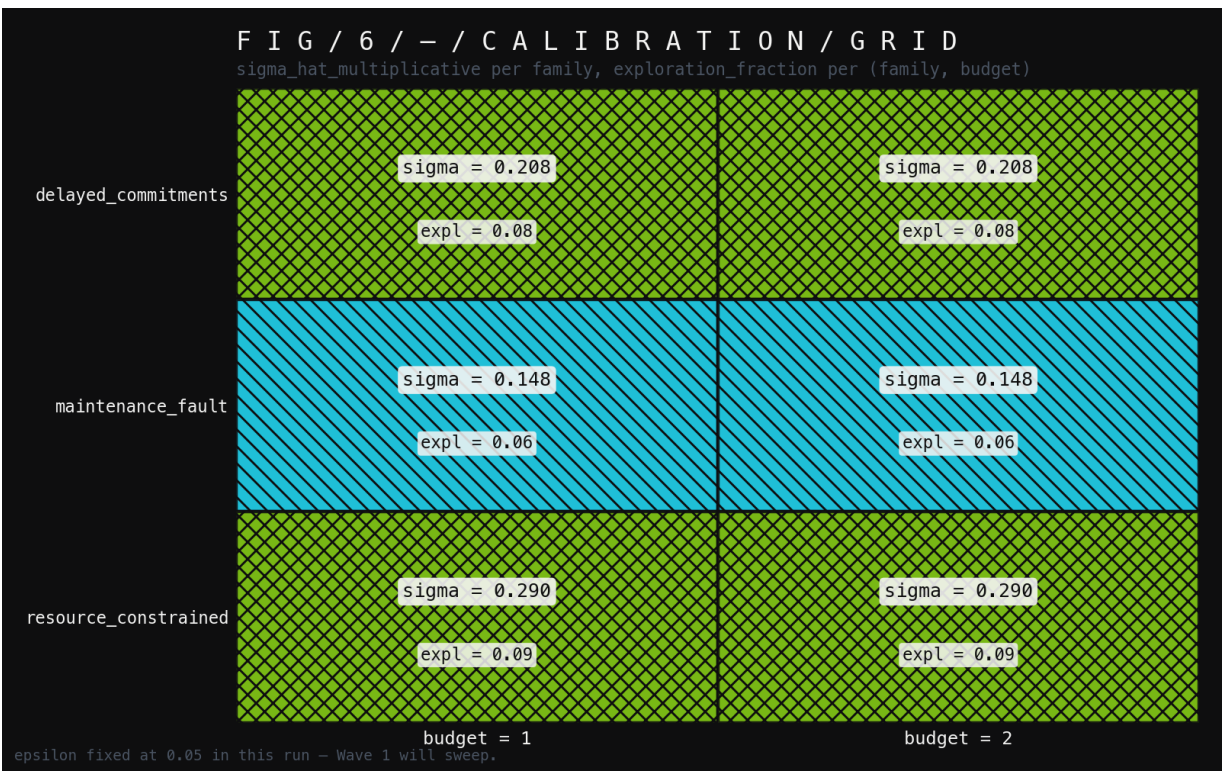


Figure 6. fig6 calibration grid dark.